

AI Insurance Claim Processor

Comprehensive System Architecture & Line-by-Line Code Documentation

1. System Architecture Overview

The application is an AI-powered Insurance Claim Processing system. It extracts insurance terms directly from uploaded PDF policies using LLMs and verifies user claims against defined coverage rules before saving results into a MySQL database.

1. Frontend Layer

HTML5 interface (`index.html`) styling and asynchronous form submission handler (`script.js`).

2. Backend Service

FastAPI REST server (`main.py`) coordinating PDF parsing (`pdf\_reader.py`) and AI analysis.

3. AI & Data Layer

Groq LLaMA integration (`groqk.py`) for structured JSON extraction and MySQL persistence (`database.py`).

2. Database Interface DATABASE.PY

This module handles persistent relational storage, opening direct connections to MySQL to record evaluated insurance claims.

Line	Code snippet	Technical Line Explanation
1	<code>import mysql.connector</code>	Imports MySQL Connector Python driver enabling SQL protocol communication.
4	<code>def save_claim(customer_name...</code>	Defines entry function accepting claim metadata and assessment result string.
6-12	<code>connection = mysql.connector.con nect(...)</code>	Establishes database socket connection to local MySQL server at port 3306.
14	<code>cursor = connection.cursor()</code>	Creates database cursor instance to execute SQL statements within the session.
16-20	<code>query = """INSERT INTO claim s..."""</code>	Constructs parameterized SQL query string using %s placeholders to prevent SQL injection.
22-27	<code>values = (customer_name, ...)</code>	Bundles record values into a Python tuple matching positional query parameters.
30	<code>cursor.execute(query, values)</code>	Executes parameterized SQL command safely passing data tuple.
32	<code>connection.commit()</code>	Commits database transaction to permanently save row additions.
34-35	<code>cursor.close() connection.close()</code>	Releases database resources and returns socket connection to pool.
37	<code>print("Claim saved to MySQL!")</code>	Logs operational confirmation message to standard output console.

3. Standalone PDF Reader PDF_READER.PY

A modular utility designed to extract plain text string data from binary PDF document files.

Line	Code snippet	Technical Line Explanation
1	<code>from pypdf import PdfReader</code>	Imports PDF document parser library class.
4	<code>def extract_text_from_pdf(...)</code>	Declares reusable function receiving file path string.
6	<code>reader = PdfReader(file_path)</code>	Initializes PDF stream reader with designated file.
8	<code>text = ""</code>	Instantiates empty text buffer variable for accumulated page text.
10-11	<code>for page in reader.pages: text += page.extract_text() or ""</code>	Iterates over all PDF pages, extracting readable text with a fallback empty string for unreadable pages.
13	<code>return text</code>	Returns full aggregated document body text to caller.

4. Prototyping & Scripting GROQK.PY

Standalone test script used to prototype PDF extraction and structured JSON extraction via Groq API prior to full FastAPI integration.

Line	Code snippet	Technical Line Explanation
1-5	<code>from groq import Groq from pypdf import ... import os, json</code>	Imports dependencies: Groq LLM client, PyPDF parser, environment loader, and standard utility packages.
7	<code>load_dotenv()</code>	Loads environment variables from <code>.env</code> file into OS environment.
9-17	<code>client = Groq(api_key=...)</code>	Instantiates Groq API client using retrieved environment secret (duplicated lines 9-17 in script).
20-29	<code>def extract_text_from_pdf(...)</code>	Defines inline helper function extracting full string context from target PDF.
32-36	<code>text = extract_text_from_pdf(...) print(text)</code>	Reads policy document file <code>sample_insurance_policy.pdf</code> and prints content to console.
40-69	<code>response = client.chat.completions.create(...)</code>	Executes LLM chat completion using system instructions demanding structured JSON format output.
72-76	<code>ai_response = response.choices[0]...</code>	Extracts message string containing model generated policy analysis output.
84	<code>claim_data = json.loads(ai_response)</code>	Parses raw JSON payload string into structured Python dictionary for verification.

5. User Interface Document INDEX.HTML

The Semantic HTML5 webpage presenting the customer-facing input form.

Line	Code snippet	Technical Line Explanation
1-11	<code><!DOCTYPE html> ... <link...></code>	Standard HTML5 document declaration, head section, meta viewport tag, and stylesheet reference.
13-22	<code><div class="container"> ... <h1></code>	Main layout container and card wrapping page titles and descriptive elements.
24-34	<code><form id="claimForm"> <input id="customerName"></code>	Form declaration and text input field for customer name submission.
37-55	<code><select id="claimType" required> ... </select></code>	Drop-down selection menu specifying Medical (Med), Prescription (Pres), or Dental claim types.
58-76	<code><input type="number" id="claimAmount"></code>	Numeric currency input restricting negative numbers (min="0") with step precision.
77-83	<code><input type="file" id="policyFile" accept=".pdf"></code>	File chooser restricted strictly to .pdf document uploads.
86-88	<code><button type="submit">Submit Claim</button></code>	Form submission button triggering the AJAX network event.
93	<code><div id="result"></div></code>	Empty container element where JavaScript dynamically injects processing outputs.
99	<code><script src="script.js"></script></code>	Loads external JavaScript application script.

6. Client-Side Script SCRIPT.JS

Manages client form handling, file serialization, asynchronous API communications, and DOM updating.

Line	Code snippet	Technical Line Explanation
1-3	<code>document.getElementById("claimForm").addEventListener(...)</code>	Attaches submit event listener and prevents default browser page refresh behavior.
5-10	<code>const customerName = document.getElementById(...)</code>	Reads current values from input fields and extracts uploaded PDF file handle.
12-15	<code>if (!policyFile) { ... return; }</code>	Validates PDF attachment presence prior to sending payload.
17-24	<code>const formData = new FormData(); ...</code>	Constructs a multipart/form-data payload bundling text fields and binary file.
28-31	<code>const response = await fetch("http://127.0.0.1:8000/claims", ...)</code>	Dispatches asynchronous HTTP POST request to FastAPI backend server.
37-51	<code>result.innerHTML = `<h2>Claim Assessment</h2>...`</code>	Renders formatted response content returned from server API onto user web interface.
52-58	<code>catch (error) { ... }</code>	Catches exceptions and updates UI with user-friendly error state message.

7. Core Server Backend MAIN.PY

The central FastAPI server orchestrating CORS, PDF ingestion, LLM processing, business logic evaluation, and database interaction.

Line	Code snippet	Technical Line Explanation
1-7	<pre>from fastapi import FastAPI... from database import save_claim</pre>	Imports web framework components, Groq SDK, PDF parser, environment tools, and internal database module.
14-20	<pre>load_dotenv() api_key = os.getenv("GROQ_API_KEY")</pre>	Loads local environment variables and raises runtime ValueError if API key is missing.
27-38	<pre>app = FastAPI() app.add_middleware(CORSMiddleware...)</pre>	Initializes FastAPI instance and configures CORS middleware allowing web clients from any origin.
45-53	<pre>def extract_text_from_pdf(file_path):</pre>	Reads PDF file pages sequentially and constructs a contiguous plain text string.
60-91	<pre>def analyse_policy(text):</pre>	Sends policy text to Groq LLM API asking for strict structured JSON output defining financial coverage limits.
98-135	<pre>def check_claim(claim_type, claim_amount, policy):</pre>	Executes rule-based business logic evaluating requested claim amount against extracted AI limits.
142-148	<pre>@app.post("/claims") async def create_claim(...)</pre>	Defines asynchronous HTTP POST endpoint processing multipart form submissions containing file uploads.
159-163	<pre>file_path = f"uploaded_..." with open(file_path, "wb") as f:...</pre>	Saves uploaded PDF stream asynchronously onto disk local filesystem as temporary file.
170-179	<pre>policy_text = extract_text... policy = analyse_policy(...)</pre>	Parses temporary PDF file text and extracts numerical policy rules using Groq LLM.
187-197	<pre>result = check_claim(...) save_claim(...)</pre>	Evaluates claim validity and persists user request and decision status into MySQL database.
204-211	<pre>return { "status": result, ... }</pre>	Returns complete result payload object back to web client formatted as JSON.

